LAB - DAY 7

Higher National Diploma in Network Engineering, Year 2
BSc (Hons) Ethical Hacking & Network Security, Year 2

Reading headers and locking down a database



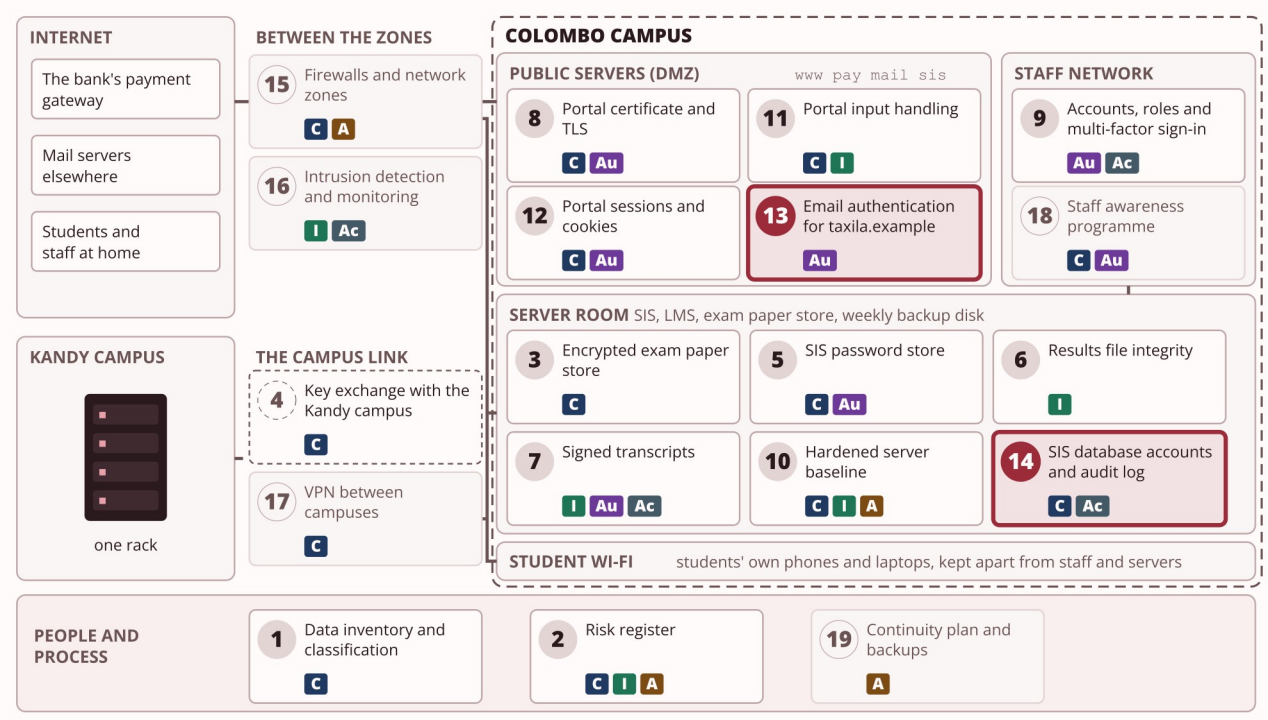
Adds to the register: 13 14

Floor tier: every step

Full tier: recommended

Niranga Dharmaratna
School of Computing & Engineering
National Institute of Business Management

The security register



TODAY

13 Email authentication for taxila.example

14 SIS database accounts and audit log

KEY

- Added today
- Added earlier
- Concept only
- Not yet added

IT

HEAD OF IT

Our mail server held twenty copies of a new "Registrar" message. Tell me where it came from, and prove it from the header.

The Head of IT, at the start of the lab.

Before you start

```
# both tiers: download the provided files to your module folder
suspect-header.txt  grants.txt  query-log.txt
# full tier: also taxila-db.sql, then take the snapshot first
snapshot name: day-7-start
```

No evidence pack today. Keep your header notes and grants, and write the two register lines of step 8 for your group.

SEGMENT 1 - 40 MIN

Analyse a suspicious header

Both tiers

Trace the quarantined header

1 Open the header; put the received lines in order, oldest first.

Expected: Taxila's server got it from `203.0.113.45`, outside Taxila.

Floor path: `header-trail`, on both tiers.

2 Mark the tells; write the true path, oldest first.

Expected: Three tells: the results, the reply-to, the origin.

```
$ grep -A3 "^Received" suspect-header.txt
$ grep -A4 "^Authentication" suspect-header.txt
```

Never paste a real header into an online checker. A real header names real people and servers.

What Taxila's server found

```
Authentication-Results: mail.taxila.example;
  spf=fail (203.0.113.45 is not a permitted sender)
  dkim=none (message not signed);
  dmarc=fail (p=QUARANTINE) header.from=taxila.example
From: "Registrar, Taxila College" <registrar@taxila.example>
Reply-To: <registrar@sis-taxila.example>
```

The from line claims Taxila. Nothing backs it, and a reply goes to a look-alike.

SEGMENT 2 - 55 MIN

Scope a database account

Both tiers

Start the database and build the SIS copy

3 Start MariaDB, then run the provided script.

Expected: `taxila-db is ready: 6 students, 12 grades`

```
$ sudo systemctl start mariadb
$ sudo mariadb < taxila-db.sql
status
taxila-db is ready: 6 students, 12 grades
```

Create the portal's account

4

Create `taxila-portal`; grant only what the portal needs.

Expected: `SHOW GRANTS` lists three lines.

Floor path: Mark `grants.txt` Part A, then check Part B.

```
$ sudo mariadb
CREATE USER 'taxila-portal'@'localhost' IDENTIFIED BY '...';
GRANT SELECT ON `taxila-db`.students TO 'taxila-portal'@'localhost';
GRANT SELECT, INSERT, UPDATE ON `taxila-db`.grades TO 'taxila-portal'@'localhost';
SHOW GRANTS FOR 'taxila-portal'@'localhost';
```

The database name has a hyphen, so it goes inside backticks in every statement.

Test it as the portal

5 Sign in as `taxila-portal`; read, update, then try a drop.

Expected: Read and update work; the drop is refused.

```
$ mariadb -u taxila-portal -p taxila-db
UPDATE grades SET mark = 64 WHERE ... ;   Query OK, 1 row affected
DROP TABLE grades;
ERROR 1142 (42000): DROP command denied to user 'taxila-portal'@'localhost'
```

A refusal is the result you want. You are the defender testing your own control.

SEGMENT 3 - 30 MIN

Turn on and read the audit log

Both tiers

Turn on and read the log

6 Turn on the general query log; run an allowed and a refused statement.

Expected: `@@general_log` *shows* `1`*.*

7 Read the end of `kali.log`.

Expected: Both statements, with the same connection number.

Floor path: `query-log.txt` and its three questions.

```
SET GLOBAL general_log = 'ON';
$ sudo tail -n 20 /var/lib/mysql/kali.log
    41 Query  SELECT student_id, module, mark FROM grades ...
    41 Query  DROP TABLE grades
```

The log records what was asked, as it arrives. It does not say what was refused: compare with the grants.



DATA PROTECTION OFFICER

If the portal's password leaks tonight, what could it reach, and would we know?

The Data Protection Officer, reading your grants and your log.

SEGMENT 4 - 25 MIN

Prove it, break it, reset and write up

Both tiers

Prove it, then break it

PROVE IT

- The header: `spf=fail`, `dkim=none`, `dmarc=fail`
- Its true origin, `203.0.113.45`, not Taxila
- The portal's account: reads and writes work; a drop is **refused**

× BREAK IT

- Grant `ALL PRIVILEGES ON *.*`; drop `throwaway.practice`
- `SHOW GRANTS` now begins with every right
- `REVOKE ALL PRIVILEGES ON *.*`; the three lines return

Reset, and the rules

RESET

- Keep your notes from steps 2, 7 and 8
- Full tier: restore `day-7-start`; MariaDB is off again
- Floor tier: keep your marked `grants.txt`

RULES OF ENGAGEMENT

- No tool in this lab can affect anyone else's system
- Only your own machine, your `taxila-db` and the provided files

Questions & Discussion

Next: Day 8, networks: threats and defences.



Disclaimer & Copyright

The information in this presentation is for general teaching purposes only. It is not comprehensive and is not professional advice; any reliance you place on it is at your own risk. I aim to keep it accurate, but I make no representations or warranties of any kind about its completeness, accuracy, reliability or suitability.

Security tools, standards and laws change. Versions, editions and commencement dates in this presentation were checked at the time of writing; check the current documentation before relying on them.

This presentation may refer to third-party websites or content. I do not endorse their views or information, and I am not responsible for their content.

Taxila College is fictional. Its campuses, people, systems and incidents were invented for teaching. Product and tool names, for example Kali Linux, VirtualBox, CyberChef and Wireshark, belong to their owners; this material is not affiliated with or endorsed by any of them.

Practise only on your own virtual machine, the module's provided files and application, and the portal's demos. Testing a system you do not own, without written permission, is an offence under Sri Lanka's Computer Crime Act, No. 24 of 2007.

© Niranga Dharmaratna, School of Computing & Engineering, National Institute of Business Management. Teaching content under CC BY-NC-SA 4.0; code and demonstrations under MIT; Open Sans under the SIL Open Font Licence.

Revision c3aa416